Principles of Programming Languages

in a nutshell

© *2023 CSE Arşiv, Free content.*

INCOMPLETE

Principles of Programming Languages in a nutshell

—

Contents

Contents **2**
Preface 3
PRELIMINARIES **4**
Reasons for Studying Principles of Programming Languages 4
Language Evaluation Criteria 4
Readability 4
Writability 5
Reliability 5
Cost 5
Language Categories 5
Implementation Methods 6
Compilation 6
Pure Interpretation 7
Describing Syntax **7**
Regular Expressions 8
BNF (Backus-Naur Form) Notation 8
Extended BNF 8
A Grammar for a Small Language 8
Lexical and Syntax Analysis **9**
Lexical Analysis (Scanning) 9
Syntax Analysis (Parsing) 9
LL Parsers 10
Example 10
Parse Table 11

Recursive Descent Parser..... 11
Names, Bindings, and Scopes..... 12
Binding..... 12
Lifetime..... 12
Scope..... 13
Static Scope..... 13
Dynamic Scope..... 13
Referencing Environments..... 14
Data Types..... 14
Dangling Pointer Problem..... 15
Tombstone..... 15
Locks-and-keys..... 15
Heap Management..... 16
Reference Counters..... 16
Mark-Sweep..... 17
Expressions and Assignment Statements..... 17
Arithmetic Expressions..... 17
Functional Side Effects..... 17
Referential Transparency..... 18
Overloaded Operators..... 18
Type Conversions..... 18
Relational and Boolean Expressions..... 18
Statement-Level Control Structures..... 19
User-Located Loop Control Mechanisms..... 19

Preface

Notları tamamlayamadığım için üzgünüm. Notlarda ya da slaytlarda anlayamadığınız yer olursa kitaptan okuyabilirsiniz o bölümü, biraz daha detaylı açıklanmış oluyor. Bu ders birçok kişi için gereksiz gibi gözükse de mülakatlarda işinize yarayabileceğini düşündüğüm programlama dilleri hakkında genel konseptler hakkında bilgiler içeriyor.
Umarım bu yarım yamalak notların da çalışmalarınıza faydası olur.

Hepinize başarılar.

PRELIMINARIES

Reasons for Studying Principles of Programming Languages

- Increased capacity to express ideas
- Improved background for choosing appropriate languages
- Increased ability to learn new languages
- Better understanding of significance of implementation
- Better use of languages that are already known
- Overall advancement of computing

Language Evaluation Criteria

Readability

The ease with which programs can be read and understood.

Characteristics that contribute to Readability:

- Overall simplicity
- Orthogonality: A programmer can design a solution to a complex problem after learning only a simple set of primitive constructs.

  - **Example:** *Adding two 32-bit integers residing in memory or registers, and replacing one of them with the sum:*
    - IBM (Mainframe) Assembly language has two instructions:
      - **A** Register1, MemoryCell1
        - Register1 ← contents(Register1) + contents(MemoryCell1)
      - **AR** Register1, Register2
        - Register1 ← contents(Register1) + contents(Register2)
    - Intel x86-64 Assembly language has one instruction:
      - **addq** operand1, operand2
        - Operand2 ← contents(operand1) + contents(operand2)
        - Here, either operand can be a register or a memory cell

  Intel instruction design is orthogonal.
  C is not an orthogonal language, because

  - records(structs) can be returned from functions but arrays cannot
  - a member of a structure can be any type but not void or structure of the same type
  - a member of an array can be any type but not void or function
- Control statements
- Data types and structures

- Syntax Considerations
    - Identifier Forms: restricting identifier length is bad for readability.
    - Special Words: Readability is increased by special words (e.g., begin, end, for).

Writability

The ease with which a language can be used to create programs.

Characteristics that contribute to writability:

- **Simplicity and orthogonality:** Few constructs, a small number of primitives, a small set of rules for combining them
- **Support for abstraction:** The ability to define and use complex structures or operations in ways that allow details to be ignored
    - **Process abstraction:** You define a subprogram, then use it by ignoring how it actually works. e.g. sort algorithm
    - **Data abstraction:** As an example of data abstraction, a tree can be represented more naturally using pointers in nodes.
- **Expressivity:** Strength and number of operators and predefined functions
    - count++; is more convenient and expressive than
    - count = count + 1;

Reliability

It performs to its specifications under all conditions.

- Type checking
- Exception handling
- Aliasing
    - Presence of two or more distinct referencing methods for the same memory location

Cost

Trade-off between compilation cost and execution cost. If expensive or runs only on expensive hardware it will not be widely used

Language Categories

- **Imperative**
    - A program consists of a sequence of statements and the execution of each statement changes the machine state
    - Statement1;
      Statement2;

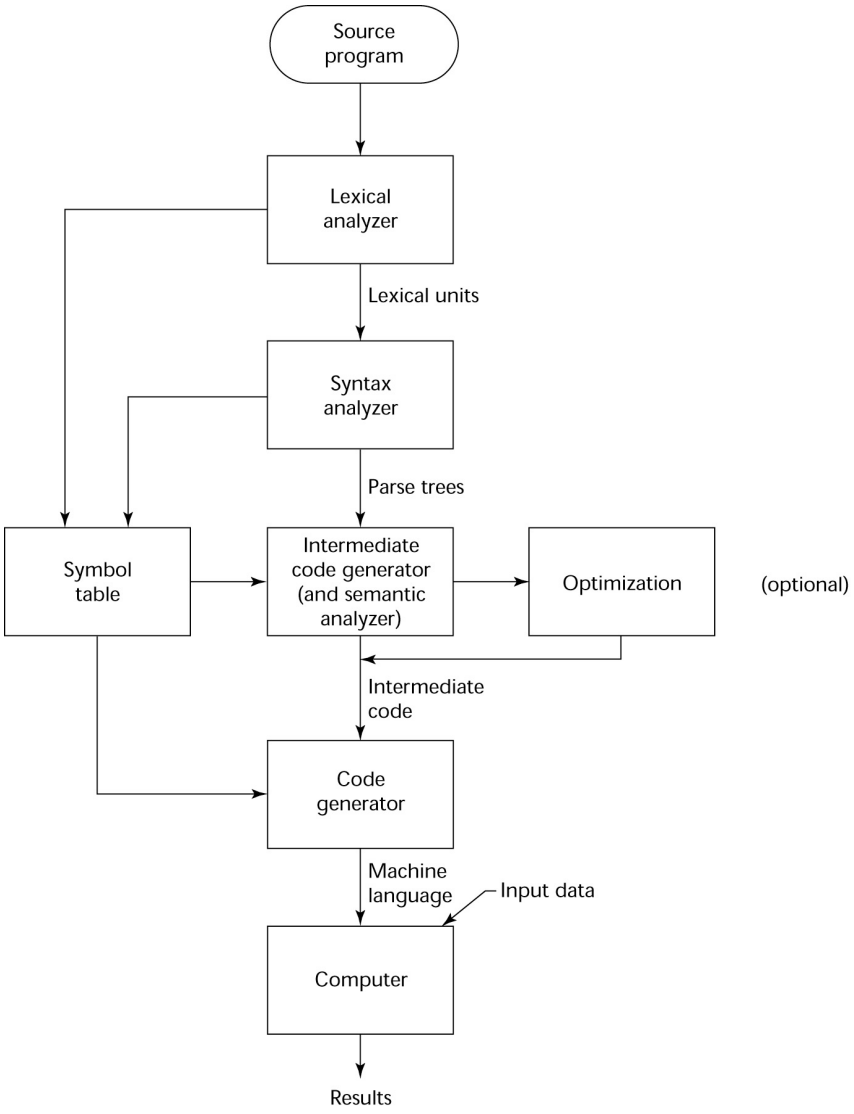
- **Functional**
    - Looks at the program as a set of functions that produce one result or another.
    - function(function1, function2, … (data))
- **Rule-based (declarative)**
    - Execute checking to see if a particular condition is true and if so, perform the appropriate actions.
    - condition1 –> action1
- **Object-Oriented**
    - Data structures and algorithms support data abstraction and allow programmer to use data in a fashion that closely represents its real world use.
    - Encapsulation, polymorphism, inheritance

Implementation Methods

- Compilation
- Pure Interpretation
- Hybrid Implementation Systems

Compilation

- **Lexical Analysis (aka. scanner):** converts characters in the source program into lexical units (output: token stream)
- **Syntax Analysis (aka. parser):** transforms lexical units into parse trees which represent the syntactic structure of program (output: parse tree)
- **Intermediate Code Generator:** produces a program at an intermediate level. Similar to assembly languages (output: abstract syntax tree or another intermediate form)
- **Semantics Analysis:** checks for errors that are difficult to check during syntax analysis, such as type errors
- **Optimization:** (optional) used if execution speed is more important than compilation speed (output: modified intermediate form)
- **Code generation:** machine code is generated (output: assembly)
- **Symbol table:** serves as a database of type and attribute information of each user defined name in a program. Placed by lexical and syntax analyzers, and used by semantic analyzer and code generator



Pure Interpretation

The interpreter program acts as a software simulation of a machine whose fetch-execute cycle deals with high level language program statements rather than machine instructions

Describing Syntax

- Syntax: the form or structure of the expressions, statements, and program units
- Semantics: the meaning of the expressions, statements, and program units
- Sentence: a string of characters over some alphabet
- Language: a set of sentences
- Lexeme: A lowest level syntactic unit of a language (e.g. *, sum, begin)
- Token: Category of lexemes (e.g. identifier, keyword, number, string)

- Ambiguity: A grammar is said to be ambiguous if there exists more than one leftmost derivation or more than one rightmost derivation or more than one parse tree for the given input string.
    - <expr> -> <expr> + <expr> | const (ambiguous)
    - <expr> -> <expr> + const | const (unambiguous)

Regular Expressions

- A single character
- The empty string, denoted by ε
- The concatenation of two regular expressions
    - Notation: RE1 RE2 (i.e. RE1 followed by RE2)
- The union of two regular expressions
    - Notation: RE1 | RE2
- The closure of a regular expression
    - Notation: RE*
    - * is known as the Kleene star
    - * represents the concatenation of 0 or more strings

BNF (Backus-Naur Form) Notation

- Terminals are lexemes or tokens
- A rule has a left-hand side (LHS), which is a nonterminal, and a right-hand side (RHS), which is a string of terminals and/or nonterminals
- Nonterminals are often enclosed in angle brackets
    - <if_stmt> → if (<logic_expr>) <stmt> | if (<logic_expr>) <stmt> else <stmt>

Extended BNF

- Optional parts are placed in brackets [ ]
    - <stmt_list> → <stmt> | <stmt> ; <stmt_list> (BNF)
    - <stmt_list> → <stmt> [; <stmt_list>] (EBNF)
- Alternative parts of RHSs are placed inside parentheses and separated via vertical bars
    - <expr> → <var> + <var> | <var> - <var> (BNF)
    - <expr> → <var> (+ | -) <var> (EBNF)
- Repetitions (0 or more) are placed inside braces { }
    - <ident> → letter {letter|digit}
    - <ident_list> → <identifier> {, <identifier>}

A Grammar for a Small Language

Grammar for Simple Assignment Statements:

```
<assign>    → <id> = <expr>
<id>        → A | B | C
<expr>      → <id> + <expr> | <id> * <expr> | ( <expr> ) | <id>
```

Leftmost Derivation: A = B * (A + C)

```
<assign>    => <id> = <expr>
            => A = <expr>
            => A = <id> * <expr>
            => A = B * <expr>
            => A = B * ( <expr> )
            => A = B * ( <id> + <expr> )
            => A = B * ( A + <expr> )
            => A = B * ( A + <id> )
            => A = B * ( A + C )
```

Lexical and Syntax Analysis

Lexical Analysis (Scanning)

- A lexical analyzer (scanner) is a pattern matcher for character strings
- Identifies substrings of the source program that belong together - *lexemes*
    - Lexemes match a character pattern, which is associated with a lexical category called a *token*
    - *sum* is a lexeme; its token may be *IDENT*
- Tokens of a programming language are a regular language, lexical analyzer is a deterministic finite automaton

Syntax Analysis (Parsing)

- Find all syntax errors; for each, produce an appropriate diagnostic message and recover quickly
- Produce the parse tree, or at least a trace of the parse tree, for the program
- Two categories of parsers
    - Top down - produce the parse tree, beginning at the root
        - Order is that of a leftmost derivation
        - Traces or builds the parse tree in preorder
    - Bottom up - produce the parse tree, beginning at the leaves

- Order is that of the reverse of a rightmost derivation

LL Parsers

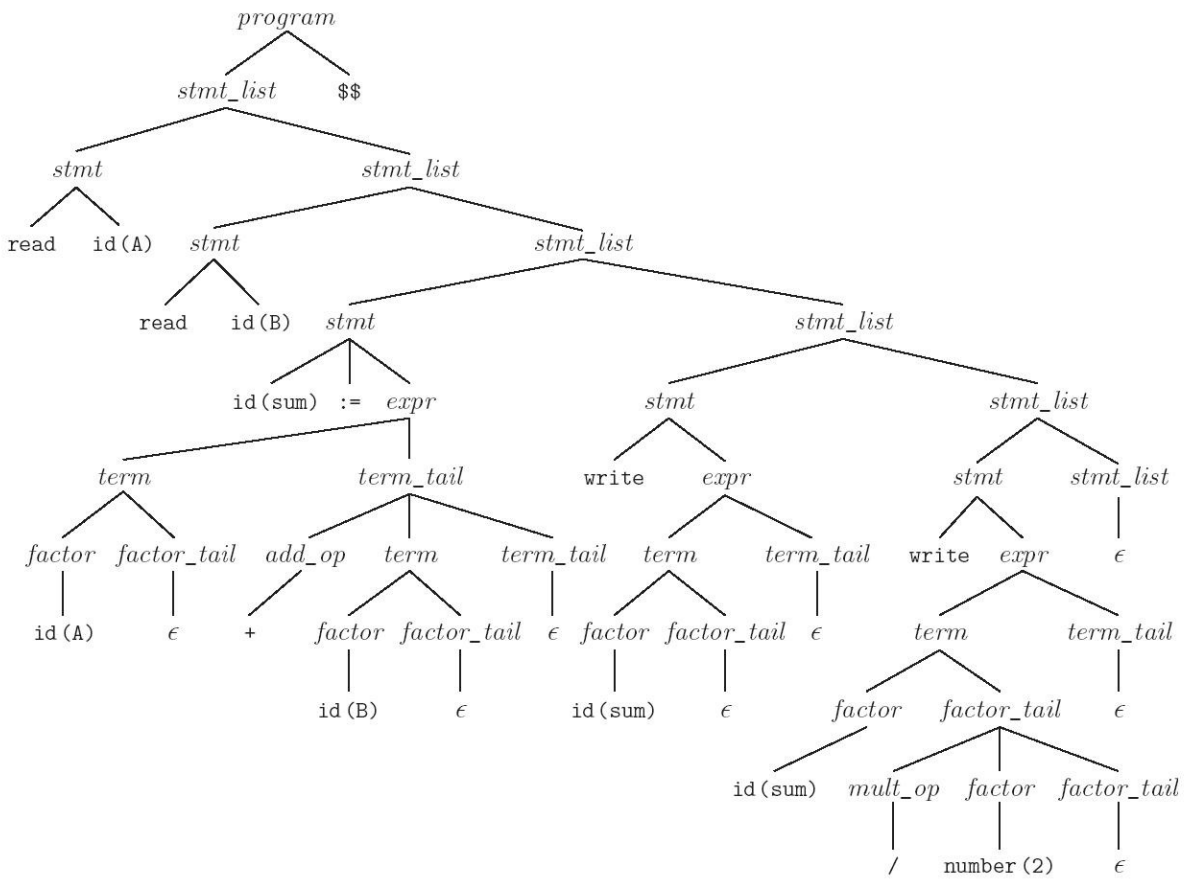
- We start at the top and predict needed productions on the basis of the current left-most non-terminal in the tree and the current input token

Example

```
 1. program   → stmt_list $$
 2. stmt_list→ stmt stmt_list
 3.            | ε
 4. stmt      → id := expr
 5.            | read id
 6.            | write expr
 7. expr      → term term_tail
 8. term_tail→ add_op term term_tail
 9.            | ε
10. term      → fact fact_tail
11. fact_tail→ mult_op fact fact_tail
12.            | ε
13. fact      → ( expr )
14.            | id
15.            | number
16. add_op    → +
17.            | -
18. mult_op   → *
19.            | /
```

Example (average program)

```
read A
read B
sum := A + B
write sum
write sum / 2
```



Parse Table

...

Recursive Descent Parser

It is a kind of Top-Down Parser. A top-down parser builds the parse tree from the top to down, starting with the start non-terminal. A Predictive Parser is a special case of Recursive Descent Parser, where no Back Tracking is required.

By carefully writing a grammar means eliminating left recursion and left factoring from it, the resulting grammar will be a grammar that can be parsed by a recursive descent parser.

Before removing left recursion	After removing left recursion
E –> E + T | T T –> T * F | F F –> (E) | id	E –> T E' E' –> + T E' | e T –> F T' T' –> * F T' | e F –> (E) | id

Names, Bindings, and Scopes

Binding

A binding is an association between an entity and an attribute, such as between a variable and its type or value, or between an operation and a symbol.

- **Static binding:** if it first occurs before run time and remains unchanged throughout program execution.
    - Type binding
        - **explicit declaration (by statement):** a program statement used for declaring the types of variables
        - **implicit declaration (by first appearance):** Means of associating variables with types through default conventions, rather than declaration statements. First appearance of a variable name in a program constitutes its implicit declaration
- **Dynamic binding:** if it first occurs at run time or can change during execution of the program
- **Dynamic Type Binding:** Type of a variable is not specified by a declaration statement
    - Less reliable: Compiler cannot check and enforce types.
    - Cost: Type checking must be done at run-time

Lifetime

- Static
    - Bound to memory cells before execution begins and remains bound to the same memory cell throughout execution
    - Efficient but reduces flexibility.
- Stack-dynamic

    - Storage bindings are created for variables when their declaration statements are elaborated .
    - Advantages: Allows recursion, same memory cells can be used for different variables
    - Disadvantages: Overhead of allocation and deallocation, subprograms cannot be history sensitive, inefficient references
- Heap-dynamic
    - Explicit Heap-dynamic
        - Allocated and deallocated by explicit directives, specified by the programmer, which take effect during execution
        - Advantage: Required for dynamic structures
        - Disadvantage: Difficult to use correctly, costly to refer, allocate, deallocate
    - Implicit Heap-dynamic
        - Allocation and deallocation caused by assignment statements
        - Advantage: highest degree of flexibility (generic code)
        - Disadvantages: Runtime overhead for allocation and deallocation, loss of error detection

Scope

The scope of a variable is the range of statements over which it is visible.

Static Scope

Search declarations, first locally, then in increasingly larger enclosing scopes, until one is found for the given name.

Dynamic Scope

References to variables are connected to declarations by searching back through the chain of subprogram calls that forced execution to this point.

```
function big() {
    function sub1() {
        var x = 7;
    }
    function sub2() {
        var y = x;
        var z = 3;
    }
    var x = 3;
}
```

big calls sub1
sub1 calls sub2
sub2 uses x

- Static scoping
  - Reference to x in sub2 is to big's x
- Dynamic scoping
  - Reference to x in sub2 is to sub1's x

Referencing Environments

The referencing environment of a statement is the collection of all names that are visible in the statement.

In a static-scoped language, it is the local variables plus all of the visible variables in all of the enclosing scopes.

In a dynamic-scoped language, the referencing environment is the local variables plus all visible variables in all active subprograms.

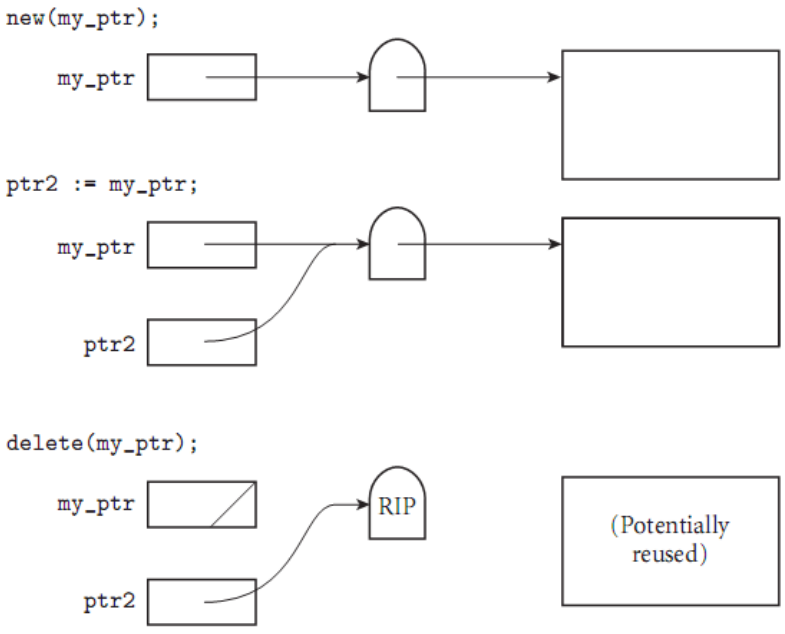
Data Types

- Data type: defines a collection of data objects and a set of predefined operations on those objects
- Descriptor: a collection of the attributes of a variable
- Object: represents an instance of a user-defined (abstract data) type

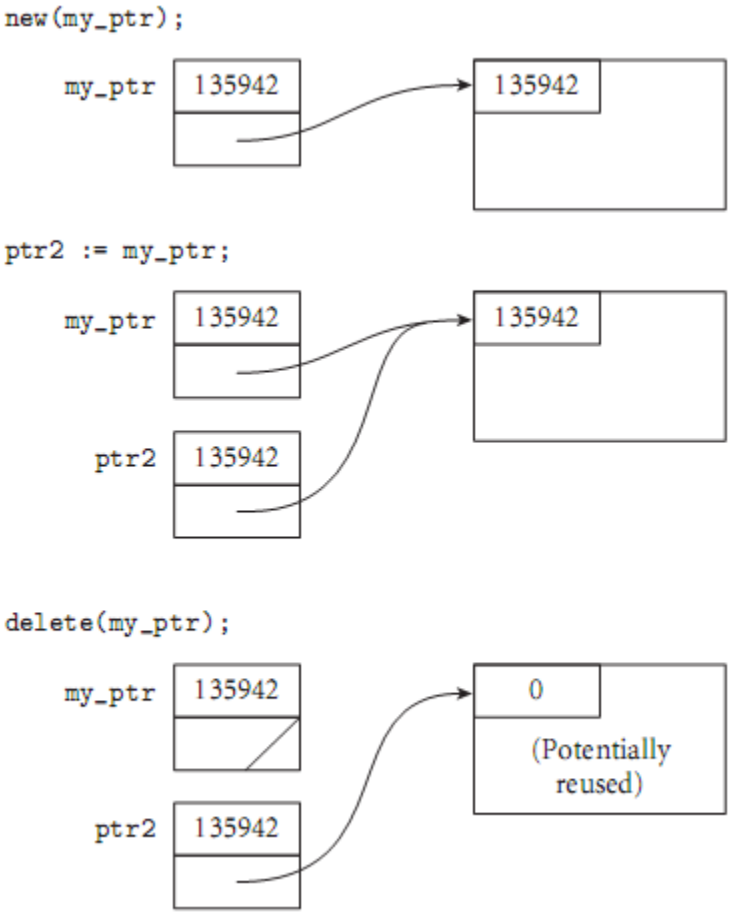
- Primitive data types: Those not defined in terms of other data types
    - Int, float, double, complex, decimal, boolean, char
    - String is primitive in some languages (Python, Java) while it is not in some others (C, C++)
- Associative Arrays: an unordered collection of data elements that are indexed by an equal number of values called keys (in Python, called as dictionaries)
- Unions: a type whose variables are allowed to store different type values at different times during execution
    - Free unions: no type checking
- Pointers: variable has a range of values that consists of memory addresses and a special value, nil
    - Dangling pointers: A pointer points to a heap-dynamic variable that has been deallocated
    - Lost heap-dynamic variable: An allocated heap-dynamic variable that is no longer accessible to the user program (garbage)

Dangling Pointer Problem

Tombstone



Locks-and-keys



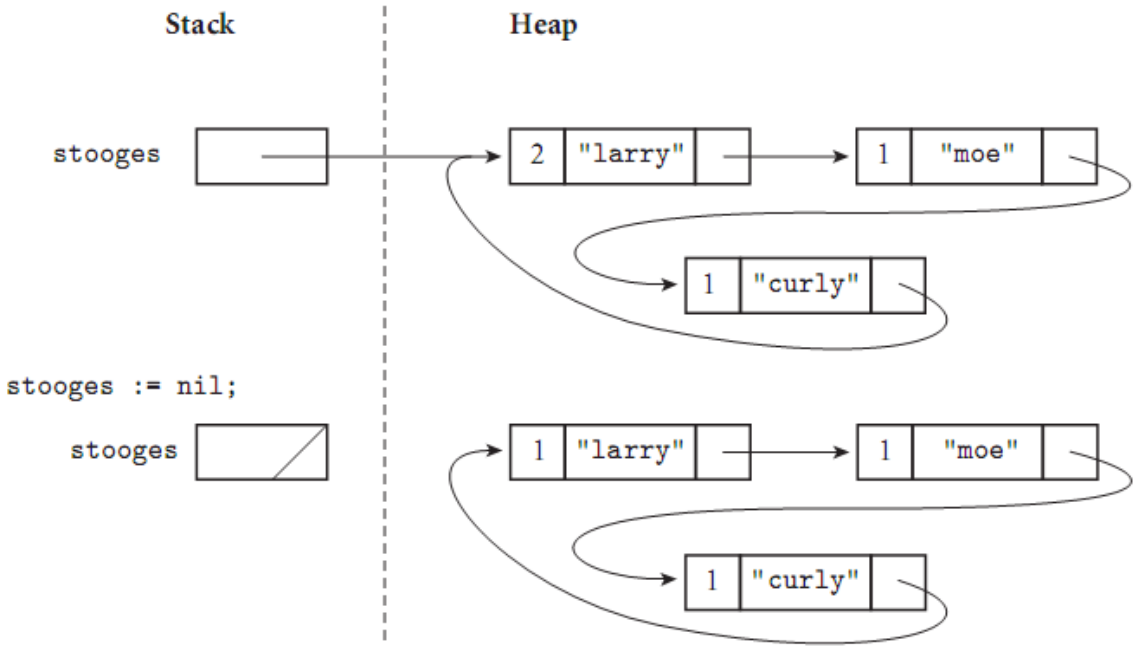
Heap Management

- **Reference counters (eager approach):** reclamation is gradual
- **Mark-sweep (lazy approach):** reclamation occurs when the list of variable space becomes empty

Reference Counters

Maintain a counter in every cell that stores the number of pointers currently pointing at the cell.

- Advantage
  - it is intrinsically incremental, so significant delays in the application execution are avoided
- Disadvantages
  - space required
  - execution time required
  - complications for cells connected circularly



Mark-Sweep

The run-time system allocates storage cells as requested and disconnects pointers from cells.

- Every heap cell has an extra bit used by collection algorithm
- All cells initially set to garbage
- All pointers traced into heap, and reachable cells marked as not garbage
- All garbage cells returned to list of available cells
- Disadvantages
    - in its original form, it was done too infrequently. When done, it caused significant delays in application execution.

Expressions and Assignment Statements

Arithmetic Expressions

- Precedence Rules: Rules for expression evaluation define the order in which "adjacent" operators of different precedence levels are evaluated
- Associativity Rule: Rules for expression evaluation define the order in which adjacent operators with the same precedence level are evaluated.
    - Left to right, except **, which is right to left

Functional Side Effects

- When a function referenced in an expression alters another operand of the expression
    - a = 10;
    - /* assume that fun changes its parameter */
    - b = a + fun(&a);
- Possible solutions
    - No two-way parameters in functions, No non-local references in functions
        - Inflexible
    - Write the language definition to demand that operand evaluation order be fixed
        - Limit some compiler optimizations

Referential Transparency

A program has the property of referential transparency if any two expressions in the program that have the same value can be substituted for one another anywhere in the program, without affecting the action of the program.

```
result1 = (fun(a) + b) / (fun(a) - c);
temp = fun(a);
result2 = (temp + b) / (temp - c);
```

If *fun* has no side effects, *result1* = *result2*. Otherwise, not, and referential transparency is violated

Semantics of a program is much easier to understand if it has referential transparency

Overloaded Operators

- Some are common (e.g., + for int and float)
    - In the prefix version (i.e., ++i), the value of i is incremented, and the value of the expression is the new value of i. So basically it first increments then assigns a value to the expression.
    - In the postfix version (i.e., i++), the value of i is incremented, but the value of the expression is the original value of i. So basically it first assigns a value to the expression and then increments the variable.
- Some are potential trouble (e.g., * in C and C++)

Type Conversions

- In most languages, all numeric types are coerced in expressions, using widening conversions.

Relational and Boolean Expressions

- One odd characteristic of C's expressions: a < b < c is a legal expression
    - Left operator is evaluated, producing 0 or 1
    - The evaluation result is then compared with the third operand
- Short Circuit Evaluation: An expression in which the result is determined without evaluating all of the operands and/or operators
    - (13 * a) * (b / 13 – 1): If a is zero, there is no need to evaluate the second part
    - C, C++ bitwise operations are not short circuits. All logic operations in Python are short circuit.

Statement-Level Control Structures

User-Located Loop Control Mechanisms